DRAFT — UNOFFICIAL — NOT FOR OPERATIONAL USE

ENTERPRISE PLAN

ODT-ENT

USAREUR-AF

ODT Workspace Enterprise Implementation Plan

Enterprise Plan

HEADQUARTERS
UNITED STATES ARMY EUROPE AND AFRICA
(USAREUR-AF)
Wiesbaden, Germany

DRAFT — NOT FOR OFFICIAL USE. FOR TRAINING PLANNING PURPOSES ONLY.

20 MARCH 2026

DRAFT — UNOFFICIAL — NOT FOR OPERATIONAL USE

DRAFT

ODT WORKSPACE ENTERPRISE IMPLEMENTATION PLAN

Version: 1.0.0 **Date**: 2026-03-12 **Owner**: ODT Engineering, USAREUR-AF G3 **Classification**: CUI **Status**: APPROVED FOR EXECUTION

EXECUTIVE SUMMARY

This plan transforms the ODT Workspace monorepo from its current state -- a functionally rich but inconsistently hardened collection of 20+ projects across 4 layers -- into a production-grade enterprise platform suitable for daily operational use by USAREUR-AF G3. The work is organized into 6 phases over 20 weeks. Every existing capability is preserved. Every phase has measurable exit criteria tied to the 46 acceptance functions, the 12 CG architectural constraints, and concrete file-level deliverables.

Current State Assessment

Dimension	Status	Gap
Products	7 apps deployed, 20+ runnable	Missing CI for 5/7 deployed apps
Pipelines	26/33 catalog pipelines loadable	Only 4 projects tested in CI
GDAP LlamaIndex	Retrieval stack code exists	0/46 acceptance gates pass (all read from empty metrics snapshot)
Security	JWT tokens in git history	No secrets manager, no BFG scrub done
Tests	7/20 projects have tests	No workspace-level test runner
CI/CD	2 apps have full CI jobs	No GDAP, MIM, Foundry, or pipeline-framework CI
Docker	27 services in compose	compose works; prod hardening absent
Observability	None	No tracing, no structured logging, no dashboards

Critical Path

```
Phase 0 (Wk 1-2)    Phase 1 (Wk 3-4)    Phase 2 (Wk 5-8)    Phase 3 (Wk 9-12)
Security + CI -----> Pipeline Harden ---> Retrieval Stack ---> Acceptance Tests
                                                                        |
                                            Phase 4 (Wk 13-16)   Phase 5 (Wk 17-20)
                                            Enterprise Ops -----> Scale + Optimize
```

PHASE 0: FOUNDATION AND SECURITY (WEEKS 1-2)

Objective: Eliminate security debt, establish CI/CD for all projects, and produce the infrastructure skeleton that every subsequent phase depends on.

CG Constraints Addressed: I (Change Framework), II (Measurability), VII (Observability), XI (Trust and Accountability)

0.1 Security Remediation (CRITICAL -- Days 1-3)

Priority: BLOCKING. No other work proceeds until JWT tokens are rotated.

0.1.1 Rotate Compromised JWT Tokens

Context: JWT tokens exist in git history. These must be rotated before any CI/CD pipeline goes live.

Tasks: 1. Audit git history for all secrets: `bash # From repo root git log --all --diff-filter=A -- '*.env' '*.pem' '*.key' trufflehog git file://. --only-verified` 2. Rotate all JWT signing keys in the production environment at `192.168.1.121`. 3. Run BFG Repo Cleaner to scrub secrets from history: `bash bfg --replace-text passwords.txt . git reflog expire --expire=now --all && git gc --prune=now --aggressive` 4. Force-push the cleaned history to origin (coordinate with all developers first). 5. Update `compose/.env` template with placeholder references (never real values).

Acceptance Tests Addressed: None directly, but a prerequisite for all P0 gates.

Success Criteria: - `trufflehog` scan returns 0 verified secrets - All production JWT tokens rotated and old tokens invalid - `compose/.env.example` exists with placeholder values; `compose/.env` is in `.gitignore`

0.1.2 Secrets Management Infrastructure

Tasks: 1. Create `config/deploy/.env.example` with every required environment variable documented: ``` # Database POSTGRES_PASSWORD= # Required. Set in CI/CD variables. CATALOG_DB_URL= # Required. PostgreSQL connection string.

External APIs UCDP_API_TOKEN= # Optional. For UCDP pipeline. ACLED_EMAIL= # Optional. For ACLED pipeline. ACLED_PASSWORD= # Optional. For ACLED pipeline.

MIM Portal MIM_WORLD_USERNAME= # Required for MIM sync. MIM_WORLD_PASSWORD= # Required for MIM sync.

LlamaIndex / GDAP GDAP_LI_PROFILE=local # local|dev|prod OPENAI_API_KEY= # Required when GDAP_LI_PROFILE != local COHERE_API_KEY= # Required when reranker=cohere `` 2. Add all .env patterns to .gitignore` at repo root. 3. Document secrets injection for GitLab CI via protected variables.

Files: - `config/deploy/.env.example` (new) - `.gitignore` (edit) - `docs/guides/secrets-management.md` (new)

0.1.3 Input Validation Hardening

Tasks: 1. Audit all FastAPI endpoints in: - `src/products/platform/apps/api/` (70 endpoints) - `src/products/chase/api/` - `src/architecture/gdap/gdap/api.py` - `src/architecture/mim/src/` (MIM admin server) 2. Ensure every endpoint uses Pydantic request models (not raw `dict`). 3. Add path sanitization to any endpoint accepting file paths: `python from pathlib import PurePosixPath def sanitize_path(user_path: str) -> str: resolved = PurePosixPath(user_path) if ".." in resolved.parts: raise ValueError("Directory traversal not permitted") return str(resolved)` 4. Add rate limiting middleware to all FastAPI apps (use `slowapi`).

Files: - `src/products/platform/apps/api/` (edit across multiple files) - `src/architecture/gdap/gdap/api.py` (edit) - `src/pipelines/pipeline-framework/pipeline_framework/` (edit -- add shared validation utils)

Success Criteria: - Every API endpoint uses typed Pydantic models for input - No endpoint accepts unsanitized file paths - Rate limiting configured at 100 req/s per IP

0.2 CI/CD Foundation (Days 3-8)

0.2.1 Workspace-Level Test Runner

Context: Only `src/pipelines/pipeline-framework`, `src/pipelines/orchestrator`, `src/pipelines/catalog/odin`, and `src/pipelines/catalog/apd` are tested in CI. 7/20 projects have any tests at all.

Tasks: 1. Add `conftest.py` to every project that has a `tests/` directory but no conftest: - `src/architecture/gdap/tests/conftest.py` - `src/architecture/mim/tests/conftest.py` - `src/products/chase/tests/conftest.py` 2. Add a root-level `pytest.ini` or extend `pyproject.toml`: `toml [tool.pytest.ini_options] testpaths = ["src"] python_files = ["test_*.py"] python_classes = ["Test*"] python_functions = ["test_*"] markers = [ "acceptance: acceptance gate tests", "integration: integration tests requiring external services",`

`"slow: tests that take > 30s", ] addopts = "--ignore=src/pipelines/catalog --tb=short -q"`

3. Create `scripts/ci/test-all.sh`: ```bash #!/bin/bash set -euo pipefail echo "=== Python Tests ===" uv run pytest src/architecture/gdap/tests/ -v --ignore=src/architecture/gdap/tests/acceptance uv run pytest src/architecture/mim/tests/ -v uv run pytest src/pipelines/pipeline-framework/tests/ -v uv run pytest src/pipelines/orchestrator/tests/ -v --ignore=tests/integration uv run pytest src/products/platform/tests/ -v uv run pytest src/foundry/foundry-platform-utils/tests/ -v

echo "=== Acceptance Tests (snapshot-based) ===" uv run pytest src/architecture/gdap/tests/acceptance/ -v || true

echo "=== Node Tests ===" # Add as projects gain test suites ```

Files: - `pyproject.toml` (edit -- add pytest config) - `scripts/ci/test-all.sh` (new) - Multiple `conftest.py` files (new)

0.2.2 Expand GitLab CI Pipeline

Context: `.gitlab-ci.yml` currently has CI for Chase, Platform, NAF, Appstore, catalog pipelines, and a deploy-all job. Missing: GDAP, MIM, Foundry tools, acceptance tests, workspace-level lint.

Tasks: 1. Add test stages for every project with tests: ```yaml test:gdap: stage: test tags: [shell, big-ubuntu] script: - cd src/architecture/gdap && uv sync --extra dev --extra llamaindex && uv run pytest tests/ -v --ignore=tests/acceptance rules: - changes: ["src/architecture/gdap/*/"] - when: manual allow_failure: true

test:mim: stage: test tags: [shell, big-ubuntu] script: - cd src/architecture/mim && uv sync && uv run pytest tests/ -v rules: - changes: ["src/architecture/mim/*/"] - when: manual allow_failure: true

test:platform: stage: test tags: [shell, big-ubuntu] script: - cd src/products/platform && uv sync --extra dev && uv run pytest tests/ -v rules: - changes: ["src/products/platform/*/"] - when: manual allow_failure: true

test:acceptance-snapshot: stage: test tags: [shell, big-ubuntu] script: - cd src/architecture/gdap && uv sync --extra dev --extra llamaindex - uv run pytest tests/acceptance/ -v --tb=long allow_failure: true rules: - changes: ["src/architecture/gdap/*/"] - when: manual ```

1. Add build+deploy jobs for GDAP and MIM: ```yaml build:gdap: stage: build tags: [shell, big-ubuntu] script:
   - cd src/architecture/gdap
   - docker build -t odt/gdap-api:${CI_COMMIT_SHORT_SHA} . rules:
   - changes: ["src/architecture/gdap/*/"]
   - when: manual allow_failure: true

deploy:gdap-api: extends: .deploy-app variables: APP_NAME: gdap-api APP_DIR: src/architecture/gdap APP_PORT: "8002" SUBDOMAIN: gdap needs: ["build:gdap"] ```

1. Add a workspace-level lint job: ```yaml lint:workspace: stage: test tags: [shell, big-ubuntu] script:
   - uv run ruff check src/ --select E,W,F --statistics allow_failure: true ```

Files: - `.gitlab-ci.yml` (edit -- expand significantly)

0.2.3 Pre-commit Hooks

Tasks: 1. Create `.pre-commit-config.yaml`: `yaml repos: - repo: https://github.com/astral-sh/ruff-pre-commit rev: v0.9.0 hooks: - id: ruff args: [--fix] - id: ruff-format - repo: https://github.com/pre-commit/pre-commit-hooks rev: v5.0.0 hooks: - id: check-added-large-files args: ['--maxkb=500'] - id: detect-private-key - id: check-merge-conflict - id: no-commit-to-branch args: ['--branch', 'main']`

Files: - `.pre-commit-config.yaml` (new)

Success Criteria: - `scripts/ci/test-all.sh` exits 0 for all non-acceptance tests - GitLab CI has test jobs for Platform, GDAP, MIM, pipeline-framework, orchestrator - Pre-commit hooks block secrets and large files - Every Python project can be linted with `ruff check`

0.3 Observability Skeleton (Days 8-10)

0.3.1 Structured Logging

Tasks: 1. Add `structlog` to workspace dependencies in root `pyproject.toml`. 2. Create shared logging config in `src/pipelines/pipeline-framework/pipeline_framework/logging.py`: ```python import structlog

def configure_logging(service_name: str, log_level: str = "INFO"): structlog.configure(processors=[structlog.contextvars.merge_contextvars, structlog.processors.add_log_level, structlog.processors.TimeStamper(fmt="iso"), structlog.processors.JSONRenderer(),], context_class=dict, logger_factory=structlog.PrintLoggerFactory(),) structlog.contextvars.bind_contextvars(service=service_name) `` 3. Integrate into GDAP API (src/architecture/gdap/gdap/api.py), Platform API (src/products/platform/apps/api/), and Chase API (src/products/chase/api/main.py`).

0.3.2 Health Check Standardization

Tasks: 1. Every FastAPI service must expose `GET /health` returning: `json {"status": "healthy", "service": "<name>", "version": "<git-sha>", "uptime_seconds": 123}` 2. Audit and fix existing health checks in `compose/docker-compose.yaml` (lines 40-43 reference port 8000 generically -- each service must check its own port).

Files: - `src/pipelines/pipeline-framework/pipeline_framework/logging.py` (new) - `src/pipelines/pipeline-framework/pipeline_framework/health.py` (new) - `compose/docker-compose.yaml` (edit health checks) - `src/architecture/gdap/gdap/api.py` (edit) - `src/products/platform/apps/api/` (edit) - `src/products/chase/api/main.py` (edit)

Success Criteria: - All API services emit JSON-structured logs - `curl localhost:<port>/health` returns valid JSON for every service in compose - Compose health checks are service-specific (not generic port 8000)

Phase 0 Exit Criteria

Criterion	Measurement	Target
Secrets in git history	`trufflehog` scan	0 verified secrets
JWT tokens rotated	Production validation	All old tokens rejected
CI test coverage	Projects with CI test jobs	>= 6 projects
Pre-commit hooks	`pre-commit run --all-files`	Exit 0
Health endpoints	`curl /health` per service	100% services respond
Structured logging	Log format validation	JSON output on all APIs

PHASE 1: CORE PIPELINE HARDENING (WEEKS 3-4)

Objective: Make the 5-stage ingestion pipeline (Raw -> Clean -> Identity -> Ontology -> Serve) production-reliable, expand test coverage, and begin decomposing oversized files.

CG Constraints Addressed: III (Understand/Adapt/Integrate), V (Never Fight the Last War), VI (New Forms of Mass), VIII (Convergence Over Duplication)

1.1 Pipeline Framework Hardening

1.1.1 Shared Pipeline Base Class

Context: `src/pipelines/pipeline-framework/pipeline_framework/` is the shared library. All 33 catalog pipelines depend on it. Currently, each pipeline implements its own `main.py` with varying patterns.

Tasks: 1. Review and standardize the pipeline base class to enforce: - Typed configuration via Pydantic - Structured logging (from Phase 0) - Error handling with categorized exceptions (transient vs. permanent) - Metrics emission (pipeline_run_duration_seconds, rows_processed_total, rows_failed_total) 2. Create `pipeline_framework/metrics.py` for Prometheus-compatible metric emission. 3. Add retry logic for transient failures (network, API rate limits).

Files: - `src/pipelines/pipeline-framework/pipeline_framework/` (edit multiple files) - `src/pipelines/pipeline-framework/pipeline_framework/metrics.py` (new) - `src/pipelines/pipeline-framework/pipeline_framework/retry.py` (new)

1.1.2 Orchestrator Stabilization

Context: `src/pipelines/orchestrator/orchestrator/` contains 26 modules including the 5-stage pipeline logic (`clean.py`, `identity.py`, `ontology.py`, `monitor.py`).

Tasks: 1. Add error boundaries around each stage in the `Makefile` `stages` target so one stage failure does not block subsequent stages. 2. Add idempotency checks: re-running a stage on unchanged data should be a no-op. 3. Create `src/pipelines/orchestrator/tests/test_stages.py` with unit tests for each stage function.

Files: - `src/pipelines/orchestrator/orchestrator/clean.py` (edit) - `src/pipelines/orchestrator/orchestrator/identity.py` (edit) - `src/pipelines/orchestrator/orchestrator/ontology.py` (edit) - `src/pipelines/orchestrator/orchestrator/monitor.py` (edit) - `src/pipelines/orchestrator/tests/test_stages.py` (new)

1.2 Large File Decomposition

Context: 10 files exceed the 500-line limit. The largest is the platform API at ~75KB.

1.2.1 Platform API Decomposition

Tasks: 1. Identify the largest files in `src/products/platform/apps/api/`. 2. Split into domain-bounded modules: - `api/routes/doctrine.py` -- doctrine query endpoints - `api/routes/catalog.py` -- catalog CRUD - `api/routes/analytics.py` -- analytics and reporting - `api/routes/admin.py` -- admin and system endpoints - `api/dependencies.py` -- shared FastAPI dependencies - `api/middleware.py` -- auth, CORS, rate limiting 3. Use FastAPI `APIRouter` for each route group. 4. Preserve all 70 endpoints exactly as they are (no behavior changes).

Files: - `src/products/platform/apps/api/` (major refactor, preserving all endpoints)

1.2.2 GDAP Retrieval Module

Context: `src/architecture/gdap/gdap/llamaindex/retrieval.py` is exactly 696 lines -- over the 500-line limit.

Tasks: 1. Extract `BM25SparseRetriever` into `src/architecture/gdap/gdap/llamaindex/sparse.py`. 2. Extract `build_property_graph_index` and `query_property_graph_triplets` into `src/architecture/gdap/gdap/llamaindex/property_graph.py`. 3. Extract `route_query` and route hints into `src/architecture/gdap/gdap/llamaindex/router.py`. 4. Keep `retrieval.py` as a facade re-exporting everything from `__all__`.

Files: - `src/architecture/gdap/gdap/llamaindex/retrieval.py` (edit -- extract) - `src/architecture/gdap/gdap/llamaindex/sparse.py` (new) - `src/architecture/gdap/gdap/llamaindex/property_graph.py` (new) - `src/architecture/gdap/gdap/llamaindex/router.py` (new) - `src/architecture/gdap/gdap/llamaindex/__init__.py` (edit)

1.3 Test Coverage Expansion

Tasks: 1. Add tests for every project currently missing them:

Project	Test Directory	Test Focus
`src/products/chase`	`tests/`	API endpoint smoke tests
`src/architecture/cda`	`tests/`	Tool output validation
`src/pipelines/catalog/gdap`	`tests/`	Dagster asset materialization
`src/foundry/ffs`	`tests/`	CLI command parsing
`src/foundry/vs-ffs`	`tests/`	VS Code extension unit tests (Jest)

1. Target: every project with Python code has at least smoke tests.

Acceptance Tests Addressed: Indirectly supports all categories by establishing test infrastructure.

Success Criteria: - Platform API split into files each under 500 lines - Retrieval module split into 4 files each under 300 lines - `src/pipelines/orchestrator/tests/test_stages.py` passes - 12/20 projects have at least one passing test - `make stages` is idempotent (re-run produces no changes)

Phase 1 Exit Criteria

Criterion	Measurement	Target
Files over 500 lines	`find src -name '*.py' | xargs wc -l | awk '$1 > 500'`	0 files
Projects with tests	Count of projects with passing test suites	>= 12
Pipeline idempotency	Run `make stages` twice, diff output	No changes on second run
Stage error isolation	Kill one stage, verify others complete	Remaining stages succeed

PHASE 2: LLAMAINDEX RETRIEVAL STACK (WEEKS 5-8)

Objective: Complete the GDAP LlamaIndex integration from ingestion through retrieval, process all 400+ doctrine PDFs, and wire the retrieval stack to the GDAP API.

CG Constraints Addressed: III (Understand/Adapt/Integrate), VI (New Forms of Mass), VIII (Convergence Over Duplication)

Acceptance Tests Addressed: A (Foundation), B (Ingestion), C (Retrieval), D (Routing)

2.1 Foundation Configuration (Week 5)

Context: `src/architecture/gdap/gdap/llamaindex/settings.py` defines `RuntimeSettings` with 3 profiles (local/dev/prod). The acceptance skeleton tests read from a `metrics_snapshot.json` file that does not yet exist with real data.

2.1.1 Settings Validation and Fail-Fast

Tasks: 1. Add startup validation to `load_runtime_settings()` in `settings.py`: - When `profile=dev` or `profile=prod`, check that `OPENAI_API_KEY` env var is set. - When `reranker=cohere`, check that `COHERE_API_KEY` is set. - Raise `SettingsValidationError` with remediation text listing the missing key. 2. Add schema version tracking: `python INDEX_SCHEMA_VERSION = "1.0.0" # Bump when chunking/embedding strategy changes` 3. Wire settings validation into GDAP API startup (`src/architecture/gdap/gdap/api.py`).

Tests Addressed: - `test_settings_load_with_safe_defaults` (A, P0) - `test_missing_required_provider_key_fails_fast` (A, P0) - `test_provider_switch_dev_vs_prod` (A, P1) - `test_index_schema_version_is_declared` (A, P0)

Files: - `src/architecture/gdap/gdap/llamaindex/settings.py` (edit) - `src/architecture/gdap/gdap/api.py` (edit) - `src/architecture/gdap/tests/test_llamaindex_settings.py` (edit -- add real test implementations)

2.1.2 Generate Initial Metrics Snapshot

Context: All acceptance tests read from `src/architecture/gdap/tests/acceptance/data/metrics_snapshot.json`. The generator script exists at `tests/acceptance/generate_metrics_snapshot.py`.

Tasks: 1. Implement `generate_metrics_snapshot.py` to: - Boot GDAP with default settings - Run a small ingestion (10 sample elements) - Build retrieval stack - Execute sample queries - Emit metrics as JSON 2. Run the generator and commit the initial snapshot. 3. Add the generator to CI as a pre-acceptance-test step.

Files: - `src/architecture/gdap/tests/acceptance/generate_metrics_snapshot.py` (edit) - `src/architecture/gdap/tests/acceptance/data/metrics_snapshot.json` (new/update)

2.2 Ingestion Pipeline Completion (Weeks 5-6)

Context: The ingestion pipeline code in `src/architecture/gdap/gdap/llamaindex/ingestion_pipeline.py` is well-structured with `DoctrineIngestionOrchestrator`, incremental reindex, QA validation, and cache. The pipeline catalog entry at `src/pipelines/catalog/gdap/` is Dagster-native.

2.2.1 PDF Ingestion at Scale

Context: 56 doctrine PDFs exist (54 US + 1 UK + 1 at root level) at `src/pipelines/catalog/gdap/doctrine_pdfs/`. The full target is 400+ PDFs.

Tasks: 1. Validate the existing PDF extraction pipeline in `src/pipelines/catalog/gdap/gdap/parse.py`: - Test with 5 representative PDFs of varying complexity - Measure extraction quality (text completeness, table handling, header/footer stripping) 2. Process all 56 existing PDFs through the pipeline: `bash cd src/pipelines/catalog/gdap for pdf in doctrine_pdfs/US/*.pdf doctrine_pdfs/*.pdf; do uv run python -c "from gdap.parse import extract_elements; extract_elements('$pdf')" done` 3. Collect remaining doctrine PDFs from source archives (NATO, UK): - Create `src/pipelines/catalog/gdap/doctrine_pdfs/NATO/` - Create `src/pipelines/catalog/gdap/doctrine_pdfs/GB/` 4. Feed all extracted `DoctrineElement` records through `DoctrineIngestionOrchestrator.run()`.

Tests Addressed: - `test_doctrine_element_maps_to_document_node_with_full_metadata` (B, P0) - `test_ingestion_pipeline_applies_transformations_in_order` (B, P0) - `test_chunking_preserves_operational_semantics` (B, P1) - `test_ingestion_cache_hits_for_unchanged_nodes` (B, P0) - `test_incremental_reindex_only_updates_changed_nodes` (B, P0) - `test_parallel_ingestion_produces_deterministic_results` (B, P1) - `test_ingestion_qa_blocks_bad_payloads` (B, P0)

Files: - `src/pipelines/catalog/gdap/gdap/parse.py` (edit -- improve extraction quality) - `src/pipelines/catalog/gdap/gdap/ingest.py` (edit -- batch processing) - `src/pipelines/catalog/gdap/doctrine_pdfs/NATO/` (new directory, populated) - `src/pipelines/catalog/gdap/doctrine_pdfs/GB/` (new directory, populated) - `src/pipelines/catalog/gdap/scripts/batch_ingest.py` (new -- orchestrated batch runner)

2.2.2 Ingestion QA Gate Implementation

Tasks: 1. Wire `run_ingestion_qa()` into every ingestion path so that: - Empty documents are quarantined (not silently dropped) - Missing critical metadata triggers a structured error log - Bad payloads are rejected before reaching the vector index 2. Add quarantine table to DuckDB: `sql CREATE TABLE IF NOT EXISTS gdap_quarantine ( unique_id VARCHAR PRIMARY KEY, reason VARCHAR NOT NULL, payload JSON, quarantined_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP );` 3. Implement the quarantine flow in `src/architecture/gdap/gdap/store.py`.

Files: - `src/architecture/gdap/gdap/store.py` (edit) - `src/architecture/gdap/gdap/llamaindex/ingestion_pipeline.py` (edit -- wire quarantine)

2.3 Retrieval Stack Implementation (Weeks 6-8)

Context: The retrieval code exists in `src/architecture/gdap/gdap/llamaindex/retrieval.py` with `build_retrieval_stack()` supporting vector, sparse (BM25), fusion (RRF), and property graph. This code is functional but not yet wired to the API or tested at scale.

2.3.1 Vector Index (Dense Retrieval)

Tasks: 1. Replace `HashEmbedding` (the deterministic test embedding) with real embeddings for non-local profiles: - `local` profile: keep `sentence-transformers/BAAI/bge-small-en-v1.5` (384-dim) - `dev` profile: `text-embedding-3-small` (1536-dim) - `prod` profile: `text-embedding-3-large` (3072-dim) 2. Wire `build_vector_index()` into the GDAP API so queries can hit the vector store. 3. Implement index persistence (save/load from disk) so indexes survive restarts: `python # In settings.py INDEX_PERSIST_DIR = Path("data/llamaindex/indexes")`

Tests Addressed: - `test_vector_relevance_on_doctrine_corpus` (C, P0) - `test_index_persistence_is_lossless` (C, P0)

2.3.2 Sparse Retrieval (BM25)

Context: `BM25SparseRetriever` is already implemented in `retrieval.py`. It handles military acronyms well because it tokenizes on `[A-Za-z0-9_]+`.

Tasks: 1. Add acronym expansion table for military-specific acronyms (MDMP, WARNO, FRAGO, etc.). 2. Test BM25 retrieval on acronym-heavy queries to verify recall. 3. Add metadata filter support to BM25 retriever (already partially implemented via `_matches_filters`).

Tests Addressed: - `test_sparse_bm25_retriever_handles_acronyms` (C, P0)

2.3.3 Fusion Retrieval (RRF)

Tasks: 1. Configure `QueryFusionRetriever` with tuned weights: - Default: `(0.6, 0.4)` favoring semantic for narrative queries - Acronym-heavy queries: `(0.3, 0.7)` favoring BM25 2. Add reranker integration for dev/prod profiles (Cohere rerank). 3. Test fusion recall against the acceptance threshold.

Tests Addressed: - `test_fusion_retriever_recall_exceeds_individual_retrievers` (C, P0)

2.3.4 Property Graph Index

Context: `build_property_graph_index()` creates entity/chunk nodes and relations (SOURCE_NATION, IN_DOCUMENT, IN_SECTION, HAS_AUTHORITY_LEVEL, HAS_PROCESS_DEPENDENCY, MENTIONS_CONCEPT, HAS_TASK). Uses LlamaIndex `SimplePropertyGraphStore`.

Tasks: 1. Populate the property graph from all ingested doctrine elements. 2. Wire property graph queries into the API for authority chain and dependency queries. 3. Implement graph traversal queries: - "Who reports to [role]?" -- traverse HAS_AUTHORITY_LEVEL relations - "What depends on [process]?" -- traverse HAS_PROCESS_DEPENDENCY relations 4. Add graph persistence alongside vector index persistence.

Tests Addressed: - `test_property_graph_traverses_authority_and_process_chains` (C, P1)

2.3.5 Query Router Integration

Context: `route_query()` in `retrieval.py` routes based on query content (SQL hints, graph hints, acronyms, quoted phrases).

Tasks: 1. Wire `route_query()` into the GDAP API query endpoint: `python @app.post("/api/query") async def query_doctrine(request: QueryRequest): route = route_query(request.query, structured_request=request.structured) if route == QueryRoute.semantic: results = stack.semantic_retriever.retrieve(request.query) elif route == QueryRoute.hybrid: results = stack.fusion_retriever.retrieve(request.query) elif route == QueryRoute.property_graph: results = query_property_graph_triplets(stack.property_graph_store, ...) elif route == QueryRoute.sql: results = execute_sql_query(request.query)` 2. Add metadata filter enforcement: - `source_nation` filter must prevent cross-nation leakage - `as_of` temporal filter for bi-temporal queries 3. Add reranker as post-retrieval step for dev/prod profiles.

Tests Addressed: - `test_router_selects_correct_retrieval_path` (D, P0) - `test_metadata_filter_enforcement` (D, P0) - `test_temporal_as_of_correctness` (D, P0) - `test_no_cross_nation_leakage` (D, P0) - `test_reranker_improves_top_k_quality` (D, P1) - `test_long_context_reorder_preserves_evidence` (D, P2)

Files (Phase 2.3 overall): - `src/architecture/gdap/gdap/llamaindex/retrieval.py` (edit) - `src/architecture/gdap/gdap/llamaindex/settings.py` (edit) - `src/architecture/gdap/gdap/api.py` (edit -- add query endpoint) - `src/architecture/gdap/gdap/llamaindex/acronyms.py` (new -- military acronym expansion) - `src/architecture/gdap/tests/test_llamaindex_retrieval.py` (edit -- real tests)

Phase 2 Exit Criteria

Criterion	Measurement	Target
PDFs processed	Count of PDFs successfully ingested	>= 56 (all existing)
DoctrineElements indexed	DuckDB row count	>= 6,092
Vector index built	Index file on disk	Exists and loads
Sparse retrieval	BM25 query for "MDMP" returns relevant results	Top-5 precision >= 0.8
Fusion recall	Fusion > max(semantic, sparse)	Recall improvement >= 5%
Property graph	Triplet count	>= 1,000 relations
Query router	Route classification accuracy	>= 95% on test queries
Acceptance metrics snapshot	`metrics_snapshot.json` populated	All Section A-D metrics present
Section A tests	pytest exit code	All 4 pass
Section B tests	pytest exit code	>= 5/7 pass
Section C tests	pytest exit code	>= 4/5 pass
Section D tests	pytest exit code	>= 4/6 pass

PHASE 3: ACCEPTANCE TEST IMPLEMENTATION (WEEKS 9-12)

Objective: Implement all remaining acceptance test sections (E through I), build the response generation layer, agent safety framework, evaluation harness, and consumer experience features.

CG Constraints Addressed: II (Measurability), IV (Echelon-Appropriate), IX (Self-Development), X (No Groupthink), XII (Leadership)

3.1 Response Quality (Section E) -- Weeks 9-10

3.1.1 Citation Engine

Tasks: 1. Create `src/architecture/gdap/gdap/llamaindex/response.py`: - Every LLM response must include inline citations `[Source: ADP 3-0, Ch. 2, Para 3]` - Extract source metadata from retrieved nodes and format as citations - If no evidence found, return safe response: "No doctrine evidence found

for this query." 2. Add structured output validation: ```python class DoctrineResponse(BaseModel): answer: str citations: list[Citation] confidence: float route_used: QueryRoute retrieval_latency_ms: float

class Citation(BaseModel): source_document: str source_section: str source_nation: str chunk_text: str relevance_score: float ``` 3. Implement streaming response for the API (SSE or WebSocket).

Tests Addressed: - `test_every_claim_has_citation` (E, P0) - `test_structured_output_matches_schema` (E, P0) - `test_no_evidence_returns_safe_response` (E, P0) - `test_streaming_response_within_sla` (E, P1) - `test_consistency_across_retries` (E, P2)

Files: - `src/architecture/gdap/gdap/llamaindex/response.py` (new) - `src/architecture/gdap/gdap/api.py` (edit -- wire response layer) - `src/architecture/gdap/tests/acceptance/test_section_e_response_quality.py` (exists -- will now pass)

3.1.2 Streaming SLA

Tasks: 1. Implement SSE streaming in the GDAP API: - First token within 2 seconds (P1 requirement) - Complete response within token budget 2. Add latency instrumentation to measure TTFB (time to first byte).

3.2 Agent Safety (Section F) -- Week 10

3.2.1 Read-Only Enforcement

Tasks: 1. Ensure all doctrine query operations are read-only: - No mutation of DuckDB doctrine tables from query paths - Use DuckDB `read_only=True` connection mode for query endpoints 2. Implement tool-use guardrails: - Agent must prefer tool-based retrieval over hallucination - If tool fails, return graceful error (not fabricated answer)

3.2.2 Session Isolation

Tasks: 1. Add session management to the GDAP API: `python class QuerySession: session_id: str user_id: str memory: list[Message] # multi-turn context filters: dict # persistent filters for this session` 2. Ensure sessions are isolated -- one user's context never leaks to another. 3. Add session storage (in-memory for dev, Redis for prod).

Tests Addressed: - `test_tools_preferred_over_hallucination` (F, P0) - `test_read_only_enforcement` (F, P0) - `test_memory_persists_across_turns` (F, P1) - `test_session_isolation` (F, P0) - `test_graceful_tool_failure` (F, P1)

Files: - `src/architecture/gdap/gdap/llamaindex/agents.py` (new) - `src/architecture/gdap/gdap/llamaindex/sessions.py` (new) - `src/architecture/gdap/gdap/api.py` (edit) -

`src/architecture/gdap/tests/acceptance/test_section_f_agents_and_safety.py` (exists -- will now pass)

3.3 Evaluation and Observability (Section G) -- Week 11

3.3.1 Retrieval Metrics

Tasks: 1. Create `src/architecture/gdap/gdap/llamaindex/evaluation.py`: ```python def compute_retrieval_metrics(query, retrieved_nodes, ground_truth_nodes): """Precision@K, Recall@K, NDCG@K, MRR.""" ...

def compute_faithfulness(response, source_nodes): """Ratio of claims supported by retrieved evidence.""" ...

def compute_relevancy(query, response): """Response relevancy to query (semantic similarity).""" ... ``` 2. Set thresholds: - Retrieval Precision@5 >= 0.7 - Faithfulness >= 0.9 - Relevancy >= 0.8

3.3.2 Regression Gate

Tasks: 1. Create `scripts/ci/acceptance-gate.sh`: `bash cd src/architecture/gdap uv run python tests/acceptance/generate_metrics_snapshot.py uv run pytest tests/acceptance/ -v --tb=long EXIT=$? if [ $EXIT -ne 0 ]; then echo "REGRESSION GATE BLOCKED: acceptance tests failed" exit 1 fi` 2. Add to GitLab CI as a blocking job on `main` branch merges.

3.3.3 Instrumentation

Tasks: 1. Add OpenTelemetry spans to: - Ingestion pipeline steps - Retrieval operations (vector, sparse, fusion, graph) - Response generation - DuckDB queries 2. Track token usage and estimated cost per query.

Tests Addressed: - `test_retrieval_metrics_above_threshold` (G, P0) - `test_faithfulness_score` (G, P0) - `test_relevancy_score` (G, P0) - `test_regression_gate_blocks_on_degradation` (G, P0) - `test_instrumentation_events_emitted` (G, P1) - `test_token_and_cost_tracking` (G, P1) - `test_observability_supports_root_cause_analysis` (G, P1)

Files: - `src/architecture/gdap/gdap/llamaindex/evaluation.py` (new) - `src/architecture/gdap/gdap/llamaindex/telemetry.py` (new) - `scripts/ci/acceptance-gate.sh` (new) - `.gitlab-ci.yml` (edit -- add regression gate job)

3.4 Operations Lifecycle (Section H) -- Week 11

3.4.1 Build/Snapshot/Restore Cycle

Tasks: 1. Implement index snapshot creation: `python def snapshot_index(index_dir: Path, snapshot_dir: Path) -> str: """Create timestamped snapshot of all LlamaIndex artifacts.""" timestamp = datetime.now(UTC).strftime("%Y%m%d_%H%M%S") snapshot_path = snapshot_dir / f"gdap_index_{timestamp}" shutil.copytree(index_dir, snapshot_path) return str(snapshot_path)` 2. Implement restore from snapshot with validation. 3. Add CLI commands to GDAP: `bash uv run gdap index snapshot --output /backups/ uv run gdap index restore --from /backups/gdap_index_20260312_120000/`

3.4.2 Embedding Model Reindex

Tasks: 1. When `INDEX_SCHEMA_VERSION` changes or embedding model changes: - Detect via manifest comparison - Trigger full reindex (not incremental) - Validate quality after reindex using canary queries 2. Add rollback capability: keep previous index version until new one passes QA.

3.4.3 Deploy Order Enforcement

Tasks: 1. Add deployment dependency checks: - DuckDB migration must complete before index build - Index build must complete before API restart - API health check must pass before serving traffic 2. Encode in `scripts/ci/deploy.sh` and Docker Compose startup order.

Tests Addressed: - `test_build_snapshot_restore_cycle` (H, P0) - `test_embedding_model_change_triggers_reindex` (H, P0) - `test_rollback_preserves_quality` (H, P1) - `test_canary_queries_validate_deployment` (H, P1) - `test_deploy_order_mismatch_prevention` (H, P0)

Files: - `src/architecture/gdap/gdap/llamaindex/lifecycle.py` (new) - `src/architecture/gdap/gdap/__init__.py` (edit -- add CLI commands) - `scripts/ci/deploy.sh` (edit -- add dependency checks)

3.5 Consumer Delight (Section I) -- Week 12

3.5.1 Response Quality Tuning

Tasks: 1. Tune prompts for military doctrine context: - Answers must be actionable, not generic - Include specific doctrine references, not vague summaries - Format for military decision-making (task/purpose/method/endstate) 2. Add confidence calibration: - High confidence (>= 0.8): direct answer with sources - Medium confidence (0.5-0.8): answer with caveats - Low confidence (<0.5): "Insufficient doctrine evidence. Consider consulting [source]."

3.5.2 UI Integration

Tasks: 1. Wire GDAP UI (`src/architecture/gdap/ui/`) to display: - Source documents with clickable citations - Query reasoning trace (which route, which indexes) - Confidence indicator 2. Ensure reproducibility: same query + same filters = same results.

Tests Addressed: - `test_fast_confident_answers` (I, P1) - `test_actionable_not_generic_responses` (I, P1) - `test_ui_shows_sources_and_reasoning` (I, P1) - `test_reproducible_with_same_filters` (I, P0) - `test_zero_trust_breakers` (I, P0)

Files: - `src/architecture/gdap/gdap/llamaindex/prompts.py` (new) - `src/architecture/gdap/ui/src/` (edit -- UI components) - `src/architecture/gdap/tests/acceptance/test_section_i_consumer_delight.py` (exists -- will now pass)

3.6 Release Readiness Gate

Tasks: 1. Wire `test_release_readiness_gate` to aggregate all section results: - Read `metrics_snapshot.json` - Verify `p0_failures_total == 0` - Verify `p1_failures_total == 0` - Verify `unresolved_critical_incidents_total == 0` 2. This is the final gate before any production release.

Files: - `src/architecture/gdap/tests/acceptance/test_release_readiness_gate.py` (already exists) - `src/architecture/gdap/tests/acceptance/generate_metrics_snapshot.py` (edit -- emit aggregate metrics)

Phase 3 Exit Criteria

Criterion	Measurement	Target
Section E tests	All 5 pass	5/5 (3 P0, 1 P1, 1 P2)
Section F tests	All 5 pass	5/5 (3 P0, 2 P1)
Section G tests	All 7 pass	7/7 (4 P0, 3 P1)
Section H tests	All 5 pass	5/5 (3 P0, 2 P1)
Section I tests	All 5 pass	5/5 (2 P0, 3 P1)
Release readiness	`test_release_readiness_gate`	Pass
Total P0 gates	Across all sections	27/27 pass
Total P1 gates	Across all sections	14/14 pass
Total P2 gates	Across all sections	2/2 pass
Regression gate in CI	GitLab job status	Blocking on main

PHASE 4: ENTERPRISE OPERATIONS (WEEKS 13-16)

Objective: Production-grade Docker deployment, monitoring, backup/restore, and operational runbooks.

CG Constraints Addressed: VII (Observability and Remediation), XI (Trust and Accountability), XII (Leadership)

4.1 Docker Production Hardening (Week 13)

4.1.1 Production Compose

Context: `compose/docker-compose.yaml` has 27 services across 5 profiles. `compose/docker-compose.prod.yaml` exists but needs hardening.

Tasks: 1. Add resource limits to all services in `compose/docker-compose.prod.yaml`: `yaml platform-api: deploy: resources: limits: cpus: '2.0' memory: 2G reservations: cpus: '0.5' memory: 512M` 2. Add log rotation: `yaml logging: driver: json-file options: max-size: "50m" max-file: "5"` 3. Add monitoring stack services: ```yaml prometheus: image: prom/prometheus:latest ports: ["9090:9090"] volumes: ["./prometheus.yml:/etc/prometheus/prometheus.yml"]

grafana: image: grafana/grafana:latest ports: ["3000:3000"] volumes: ["grafana-data:/var/lib/grafana"] `` `4. Fix the broken` src/foundry/tauri-test/Dockerfile`.

Files: - `compose/docker-compose.prod.yaml` (edit) - `compose/prometheus.yml` (new) - `compose/grafana/` (new -- provisioning dashboards) - `src/foundry/tauri-test/Dockerfile` (fix)

4.1.2 Nginx Reverse Proxy

Context: `compose/nginx/` directory exists.

Tasks: 1. Configure nginx as the unified entrypoint: - TLS termination (with self-signed certs for dev, Let's Encrypt for prod) - Rate limiting per upstream - Request/response logging - Static file caching for UI assets 2. Add security headers (HSTS, CSP, X-Frame-Options).

Files: - `compose/nginx/nginx.conf` (edit) - `compose/nginx/ssl/` (new -- cert generation script)

4.2 Monitoring and Alerting (Week 14)

4.2.1 Prometheus Metrics

Tasks: 1. Add `/metrics` endpoint to all Python APIs using `prometheus-fastapi-instrumentator`: - Request count, latency histogram, error rate - Active connections - Custom: `gdap_queries_total`, `gdap_ingestion_duration_seconds`, `gdap_index_size_bytes` 2. Configure Prometheus scrape

targets.

4.2.2 Grafana Dashboards

Tasks: 1. Create dashboards: - **ODT Overview**: All service health, request rates, error rates - **GDAP Doctrine**: Ingestion progress, query latency, index size, retrieval quality - **Pipeline Status**: Catalog refresh status, row counts, stage durations - **Infrastructure**: PostgreSQL connections, DuckDB file sizes, disk usage 2. Configure alerting rules: - Service down > 5 minutes - Error rate > 5% - Query latency p95 > 10 seconds - Disk usage > 80%

Files: - `compose/grafana/provisioning/dashboards/odt-overview.json` (new) - `compose/grafana/provisioning/dashboards/gdap-doctrine.json` (new) - `compose/grafana/provisioning/dashboards/pipeline-status.json` (new) - `compose/grafana/provisioning/datasources/prometheus.yml` (new)

4.3 Backup and Disaster Recovery (Week 14)

Tasks: 1. Automate the existing `make backup` target to run on a schedule. 2. Add GDAP-specific backups: - DuckDB database: `src/architecture/gdap/gdap.duckdb` - LlamaIndex artifacts: `data/llamaindex/` - Version manifests: `data/llamaindex/version_manifest.json` 3. Create restore runbook in `docs/guides/disaster-recovery.md`. 4. Test restore from backup weekly (automate in CI).

Files: - `scripts/ci/backup.sh` (new) - `docs/guides/disaster-recovery.md` (new)

4.4 Operational Runbooks (Weeks 15-16)

Tasks: 1. Create runbooks for common operations:

Runbook	Path	Content
Deployment	`docs/guides/deployment.md`	Step-by-step deploy for each app
Incident Response	`docs/guides/incident-response.md`	Escalation matrix, on-call procedures
Catalog Refresh	`docs/guides/catalog-refresh.md`	How to run full/partial refresh
GDAP Reindex	`docs/guides/gdap-reindex.md`	When and how to trigger reindex
Scaling	`docs/guides/scaling.md`	Horizontal scaling procedures

1. Create training materials (CG Constraint IX -- Self-Development):
2. `docs/guides/onboarding.md` -- new developer setup
3. `docs/guides/architecture-overview.md` -- system context and data flow

CG Constraint IX Compliance: Every capability has a training product in `docs/guides/`.

4.5 Drift Detection (Week 16)

Context: CG Constraint III requires a 30-day staleness flag. CG Constraint VII requires drift detection.

Tasks: 1. Create `src/architecture/gdap/gdap/temporal.py` enhancements: - Flag any `DoctrineElement` where `extracted_at` is > 30 days old and no re-extraction has occurred - Emit `gdap_stale_elements_total` metric 2. Add drift detection dashboard in Grafana showing: - Elements by staleness bucket (fresh/30d/60d/90d+) - Concept drift trends (using `ConceptDrift` model) 3. Add weekly CI job to report staleness.

Files: - `src/architecture/gdap/gdap/temporal.py` (edit) - `scripts/ci/staleness-report.sh` (new) - `.gitlab-ci.yml` (edit -- add weekly staleness job)

Phase 4 Exit Criteria

Criterion	Measurement	Target
Docker prod stack	`make compose-prod` succeeds	All 27 services healthy
Resource limits	Every service has memory/CPU limits	100%
Prometheus metrics	`/metrics` endpoint on all APIs	All Python APIs
Grafana dashboards	Dashboard count	>= 4 operational dashboards
Backup automation	Scheduled backup runs	Daily, verified weekly
Restore tested	Full restore from backup	Completes successfully
Runbook coverage	Runbooks for common operations	>= 5 runbooks
Staleness detection	Stale element reporting	Automated weekly
Tauri Dockerfile	Build succeeds	Exit 0

PHASE 5: SCALE AND OPTIMIZATION (WEEKS 17-20)

Objective: Process the full 400+ PDF corpus, optimize query performance, establish external review processes, and complete all remaining hardening.

CG Constraints Addressed: V (Never Fight Last War), VI (New Forms of Mass), X (No Groupthink), VIII (Convergence)

DRAFT

5.1 Full PDF Corpus Processing (Week 17)

Tasks: 1. Acquire remaining doctrine PDFs to reach 400+ total: - US Army doctrine publications (FM, ADP, ATP series) - NATO standardization agreements (STANAGs) - UK Joint Doctrine Publications (JDPs) 2. Batch process all PDFs: `bash # Estimated: 400 PDFs * ~50 elements/PDF = ~20,000 DoctrineElements cd src/pipelines/catalog/gdap uv run python scripts/batch_ingest.py --pdf-dir doctrine_pdfs/ --parallel 4` 3. Run full ingestion through `DoctrineIngestionOrchestrator`: - Expect ~20,000 elements producing ~100,000 chunks - Build vector index, BM25 index, property graph 4. Validate with canary queries after full ingestion.

Success Criteria: - 400+ PDFs processed - 20,000+ DoctrineElements in DuckDB - 100,000+ chunks in vector index - Property graph with 5,000+ relations - All canary queries return relevant results

5.2 Query Performance Optimization (Week 18)

Tasks: 1. Benchmark current query latency: - Target: p50 < 2s, p95 < 5s, p99 < 10s for semantic queries - Target: p50 < 500ms for BM25 queries 2. Optimize vector index: - Use HNSW parameters: `M=16`, `ef_construction=200`, `ef_search=100` - Consider Qdrant for prod (already in settings as the prod vector store) 3. Add query result caching (TTL-based, invalidated on reindex). 4. Optimize BM25: pre-compute IDF values, use inverted index. 5. Add index preloading on service startup (load into memory at boot).

Files: - `src/architecture/gdap/gdap/llamaindex/retrieval.py` (edit -- optimization) - `src/architecture/gdap/gdap/llamaindex/cache.py` (new -- query cache) - `scripts/benchmark/query_benchmark.py` (new)

5.3 Shared UI Library Activation (Week 18)

Context: `@gdap/ui` shared library exists but is not consumed by any project.

Tasks: 1. Audit `src/architecture/gdap/ui/` for reusable components. 2. Extract shared components into a package consumable by: - GDAP UI - Platform UI - ODT Appstore 3. Publish as internal npm package or use turborepo workspace references.

Files: - `src/architecture/gdap/ui/` (refactor) - `turbo.json` (edit -- add ui package) - `package.json` (edit -- workspace config)

5.4 External Review Process (Week 19)

Context: CG Constraint X requires outside review and annual rotation to prevent groupthink.

Tasks: 1. Establish peer review gates in GitLab: - All PRs to `main` require 2 approvals - At least 1 approval must be from outside the immediate team - CODEOWNERS file designating domain experts per path 2. Create architecture review board (ARB) process: - Monthly review of architectural decisions - ADR

template in `docs/adr/` - Rotation schedule for reviewers 3. Document in `docs/guides/review-process.md`.

Files: - `CODEOWNERS` (new) - `docs/adr/TEMPLATE.md` (new) - `docs/guides/review-process.md` (new)

5.5 No Single-Vendor Lock-In Validation (Week 19)

Context: CG Constraint V -- must not be locked to a single vendor.

Tasks: 1. Validate LlamaIndex provider abstraction: - `local` profile uses Ollama + sentence-transformers (open source) - `dev` / `prod` profiles use OpenAI (commercial) - Demonstrate switching from OpenAI to Azure OpenAI or Anthropic 2. Validate database portability: - DuckDB for local/dev - PostgreSQL for prod - Both use same schema contracts 3. Document vendor dependencies and alternatives in `docs/architecture/vendor-matrix.md`.

Files: - `docs/architecture/vendor-matrix.md` (new)

5.6 Final Hardening (Week 20)

Tasks: 1. Run full acceptance test suite and fix any remaining failures. 2. Run security scan: `bash # Run project security scan tooling` 3. Run load test against production-like environment: - 50 concurrent users - Mixed query workload (60% semantic, 25% hybrid, 10% graph, 5% SQL) - Sustained for 30 minutes 4. Generate final compliance report against all 12 CG constraints. 5. Cut release tag.

Files: - `scripts/benchmark/load_test.py` (new) - `docs/architecture/cg-compliance-report.md` (new)

Phase 5 Exit Criteria

Criterion	Measurement	Target
PDFs processed	Total count	>= 400
DoctrineElements	DuckDB row count	>= 20,000
Query latency p95	Benchmark results	< 5 seconds
BM25 latency p95	Benchmark results	< 500ms
Shared UI consumed	Projects using @gdap/ui	>= 2
CODEOWNERS	File exists with domain mapping	All src/ paths covered
Vendor alternatives	Documented for LLM, embedding, vector store, database	>= 2 per component
Load test	50 concurrent users, 30 min	0 errors, p95 < 5s

Criterion	Measurement	Target
CG compliance	All 12 constraints	12/12 documented and satisfied
Release readiness	`test_release_readiness_gate`	Pass

CROSS-CUTTING CONCERNS

Acceptance Test Summary

Section	Tests	P0	P1	P2	Phase
A. Foundation	4	3	1	0	2
B. Ingestion	7	4	2	0	2
C. Retrieval	5	3	1	0	2
D. Routing	6	3	1	1	2
E. Response Quality	5	3	1	1	3
F. Agents & Safety	5	3	2	0	3
G. Evaluation	7	4	3	0	3
H. Operations	5	3	2	0	3
I. Consumer Delight	5	2	3	0	3
Release Gate	1	1	0	0	3
TOTAL	**50**	**29**	**16**	**2**	

CG Constraint Compliance Matrix

#	Constraint	Phase	How Addressed
I	Change Framework	0	This plan (Why/Vision/Plan), CI/CD (Process), runbooks (Culture)
II	Measurability	0, 3	Acceptance metrics, Prometheus metrics, evaluation harness (MOP/MOE before dev)
III	Understand/Adapt/Integrate	1, 4	30-day staleness flag, drift detection, temporal queries

DRAFT

#	Constraint	Phase	How Addressed
IV	Echelon-Appropriate	3	Authority-level filtering, role-based access on query results
V	No Last War	5	Multi-vendor LLM/embedding/vector store abstraction, vendor matrix
VI	New Forms of Mass	1, 2	Decision speed via sub-5s retrieval, property graph for rapid traversal
VII	Observability and Remediation	0, 4	Structured logging, Prometheus, Grafana, drift detection
VIII	Convergence	1, 2	Single ontology (DoctrineElement), single retrieval stack, @gdap/ui shared lib
IX	Self-Development	4	Runbooks, onboarding guide, architecture overview, training materials
X	No Groupthink	5	CODEOWNERS, 2-approval PRs, external reviewer requirement, ARB rotation
XI	Trust and Accountability	0, 4	Audit logging, session tracking, secrets management, incident response
XII	Leadership	3, 4	Release readiness gate requires leadership sign-off, compliance report

Existing Features Preservation Checklist

Every product, pipeline, tool, and capability must survive. This checklist is validated at each phase gate.

Layer 1 -- Architecture: - ☐ CDA: 23 Python tools, NAF Builder, Course Portal, Hermes, Lessons, EA Ref, Compass - ☐ GDAP: knowledge graph engine, DuckDB, FastAPI API, React UI - ☐ MIM: NATO MIM 5.3 parser, 36 tests, admin UI, viewer - ☐ CG-Guidance: source documents preserved

Layer 2 -- Pipelines: - ☐ pipeline-framework: shared library, tests - ☐ Dagster orchestrator: definitions, schedules, sensors - ☐ 33 catalog pipelines (26 loadable): all `main.py` entry points work - ☐ 5-stage pipeline: Raw, Clean, Identity, Ontology, Serve - ☐ GDAP catalog: Dagster-native assets

Layer 3 -- Products: - ☐ Platform/ODE: v3.8.0, 113 tests, 70 endpoints, CLI, desktop app - ☐ Chase: ML conflict prediction, FastAPI + React + Streamlit - ☐ One-More-Brief: MDMP desktop Tauri app - ☐ ODT-Appstore: app catalog - ☐ Platform Intelligence: API + UI

Layer 4 -- Foundry: - ☐ foundry-platform-utils/odt: core library - ☐ ffs: CLI tool - ☐ vs-ffs: VS Code extension (v0.5.0) - ☐ foundry-os-lab: desktop tool - ☐ foundry-local-builder: build tool - ☐ foundry-widget-factory: widget toolkit - ☐ tauri-test: desktop test harness

Infrastructure: - ☐ Docker Compose: 27 services, 5 profiles - ☐ config/deploy/deploy.yml: 6 apps on militarydev.net - ☐ Makefile: 50+ targets all functional - ☐ GitLab CI: existing jobs preserved - ☐ DuckDB databases: ode.db, gdap.duckdb, catalog.duckdb, atis.duckdb

Ruflo Agents and Tools for Execution

Phase	Agent Type	Count	Purpose
0	`security-architect`	1	Audit git history, coordinate rotation
0	`coder`	3	CI pipeline expansion, pre-commit hooks, logging
1	`coder`	4	File decomposition, test writing, pipeline hardening
1	`reviewer`	1	Validate decomposition preserves behavior
2	`coder`	4	Settings, ingestion, retrieval, router implementation
2	`tester`	2	Acceptance test implementation, metric snapshot
3	`coder`	3	Response layer, agent safety, evaluation
3	`tester`	2	Sections E-I test implementation
4	`coder`	2	Docker hardening, monitoring stack
4	`researcher`	1	Runbook authoring
5	`performance-engineer`	1	Query optimization, load testing
5	`security-auditor`	1	Final security scan

Use hierarchical agent coordination with max 8 agents and specialized strategy for each phase.

Risk Register

Risk	Likelihood	Impact	Mitigation
JWT tokens already exploited	Medium	Critical	Rotate immediately (Phase 0 Day 1), audit access logs
400+ PDF extraction quality varies	High	Medium	Quarantine pipeline, manual review queue for low-confidence extractions
LlamaIndex API breaking changes	Medium	Medium	Pin versions in pyproject.toml, test before upgrading
Query latency exceeds SLA at scale	Medium	High	Qdrant for prod, query caching, index preloading

Risk	Likelihood	Impact	Mitigation
Team bandwidth insufficient for 20-week plan	Medium	High	Prioritize P0 gates first, defer P2 items
DuckDB concurrency limits	Low	Medium	Read-only connections for queries, single writer pattern

Dependencies and Blocking Items

```
Phase 0 ──> Phase 1 ──> Phase 2 ──> Phase 3 ──> Phase 4 ──> Phase 5
  |            |            |            |            |
  |            |            |            |            +-- Needs: all acceptance tests
passing
  |            |            |            +-- Needs: retrieval stack working
  |            |            +-- Needs: pipeline stable, files decomposed
  |            +-- Needs: CI working, secrets clean
  +-- BLOCKS EVERYTHING: security remediation
```

Within phases, tasks are parallelizable. Between phases, the dependencies above are strict.

APPENDIX A: FILE INDEX

Key files referenced in this plan, grouped by area:

GDAP Core: - `src/architecture/gdap/gdap/models.py` -- DoctrineElement and domain models - `src/architecture/gdap/gdap/api.py` -- FastAPI endpoints - `src/architecture/gdap/gdap/store.py` -- DuckDB storage - `src/architecture/gdap/gdap/temporal.py` -- bi-temporal logic - `src/architecture/gdap/gdap/alignment.py` -- cross-nation alignment

GDAP LlamaIndex: - `src/architecture/gdap/gdap/llamaindex/settings.py` -- runtime config (3 profiles) - `src/architecture/gdap/gdap/llamaindex/ingestion_adapter.py` -- DoctrineElement to Document - `src/architecture/gdap/gdap/llamaindex/ingestion_pipeline.py` -- orchestrator, QA, cache - `src/architecture/gdap/gdap/llamaindex/retrieval.py` -- vector, sparse, fusion, graph

Acceptance Tests: - `src/architecture/gdap/tests/acceptance/_skeleton.py` -- metric assertion framework - `src/architecture/gdap/tests/acceptance/test_section_a_foundation.py` - `src/architecture/gdap/tests/acceptance/test_section_b_ingestion.py` - `src/architecture/gdap/tests/acceptance/test_section_c_retrieval_architecture.py` - `src/architecture/gdap/tests/acceptance/test_section_d_routing_and_filters.py` - `src/architecture/gdap/tests/acceptance/test_section_e_response_quality.py` - `src/architecture/gdap/tests/acceptance/test_section_f_agents_and_safety.py` -

`src/architecture/gdap/tests/acceptance/test_section_g_evaluation_and_observability.py` - `src/architecture/gdap/tests/acceptance/test_section_h_operations_lifecycle.py` - `src/architecture/gdap/tests/acceptance/test_section_i_consumer_delight.py` - `src/architecture/gdap/tests/acceptance/test_release_readiness_gate.py`

Pipeline: - `src/pipelines/pipeline-framework/pipeline_framework/` -- shared library - `src/pipelines/orchestrator/orchestrator/` -- 5-stage orchestration - `src/pipelines/catalog/gdap/` -- GDAP Dagster pipeline - `src/pipelines/catalog/gdap/doctrine_pdfs/US/` -- 54 US doctrine PDFs

Infrastructure: - `compose/docker-compose.yaml` -- 27 services - `compose/docker-compose.prod.yaml` -- production overrides - `.gitlab-ci.yml` -- CI/CD pipeline - `config/deploy/deploy.yml` -- deployment manifest - `Makefile` -- 50+ workspace targets - `pyproject.toml` -- UV workspace config

Products: - `src/products/platform/` -- ODE (Platform) - `src/products/chase/` -- Chase conflict prediction - `src/products/one-more-brief/` -- MDMP planning tool - `src/products/odt-appstore/` -- app catalog - `src/products/internal/platform-intelligence/` -- analytics

APPENDIX B: WEEKLY MILESTONE CALENDAR

Week	Phase	Key Deliverable	Gate
1	0	JWT rotation complete, secrets cleaned	Security sign-off
2	0	CI expanded, pre-commit hooks, logging	CI green for all projects
3	1	Pipeline framework hardened, stages idempotent	`make stages` idempotent
4	1	Files decomposed, 12+ projects with tests	No file > 500 lines
5	2	Settings validated, metrics snapshot generated	Section A tests pass
6	2	All 56 PDFs ingested, QA gate enforced	Section B tests pass
7	2	Vector + sparse + fusion retrieval working	Section C tests pass
8	2	Router + filters + reranker complete	Section D tests pass
9	3	Citation engine, structured responses	Section E tests pass
10	3	Agent safety, session isolation	Section F tests pass
11	3	Evaluation harness, regression gate in CI	Section G + H tests pass
12	3	Consumer delight tuning, release gate	All 50 acceptance tests pass
13	4	Docker prod hardening, monitoring stack	Compose prod healthy
14	4	Grafana dashboards, backup automation	4+ dashboards live

Week	Phase	Key Deliverable	Gate
15	4	Operational runbooks, training materials	5+ runbooks published
16	4	Drift detection, staleness reporting	Automated weekly reports
17	5	400+ PDFs processed	20,000+ elements indexed
18	5	Query optimization, shared UI activated	p95 < 5s
19	5	External review process, vendor matrix	CODEOWNERS + ARB process
20	5	Load test, security scan, compliance report	CG compliance: 12/12

This plan satisfies all 12 CG USAREUR-AF architectural constraints, addresses all 50 acceptance functions (46 domain + 4 aggregate), preserves all existing features across 4 layers and 20+ projects, and provides measurable exit criteria for each of 6 phases over 20 weeks.

DRAFT